FoodBridge - Entity Relationship Diagram (ERD):

Create ERD DocumentDescription:

Create the Entity Relationship Diagram document for the FoodBridge system. This document should show the main database entities, their attributes, and relationships.

Tasks:

- Identify main entities: User, FoodDonation, FoodRequest, DeliveryTask, Notification, Report, AuditLog
- Add entity attributes with primary keys and foreign keys
- Show relationships between entities
- Add Mermaid ERD diagram
- Explain business rules and relationships

Note: This ERD is a logical domain model used for requirements and design communication.

Entity Overview:

Entity	Purpose
User	Stores donor, NGO, volunteer, and administrator account information
FoodDonation	Stores food donation details created by donors
FoodRequest:	Stores NGO requests for available food donations
DeliveryTask	Stores volunteer pickup and delivery assignments
Notification	Stores notification events for requests, approvals, pickup, and delivery updates
Report	Stores generated administrative reports
AuditLo	Stores security and activity logs

Entity Attributes:

User:

user_id, full_name, email, password_hash, role, phone, address, is_verified, created_at, updated_at

FoodDonation:

donation_id, donor_id, food_type, quantity, expiry_time, pickup_location, status, created_at, updated_at

FoodRequest:

request_id, donation_id, ngo_id, request_status, requested_quantity, message, created_at, updated_at

DeliveryTask:

delivery_id, request_id, volunteer_id, pickup_address, delivery_address, pickup_time, delivery_status, completed_at

Notification:

notification_id, user_id, related_entity, title, message, type, read_status, created_at

Report:

report_id, admin_id, report_type, date_range, total_donations, total_deliveries, generated_at

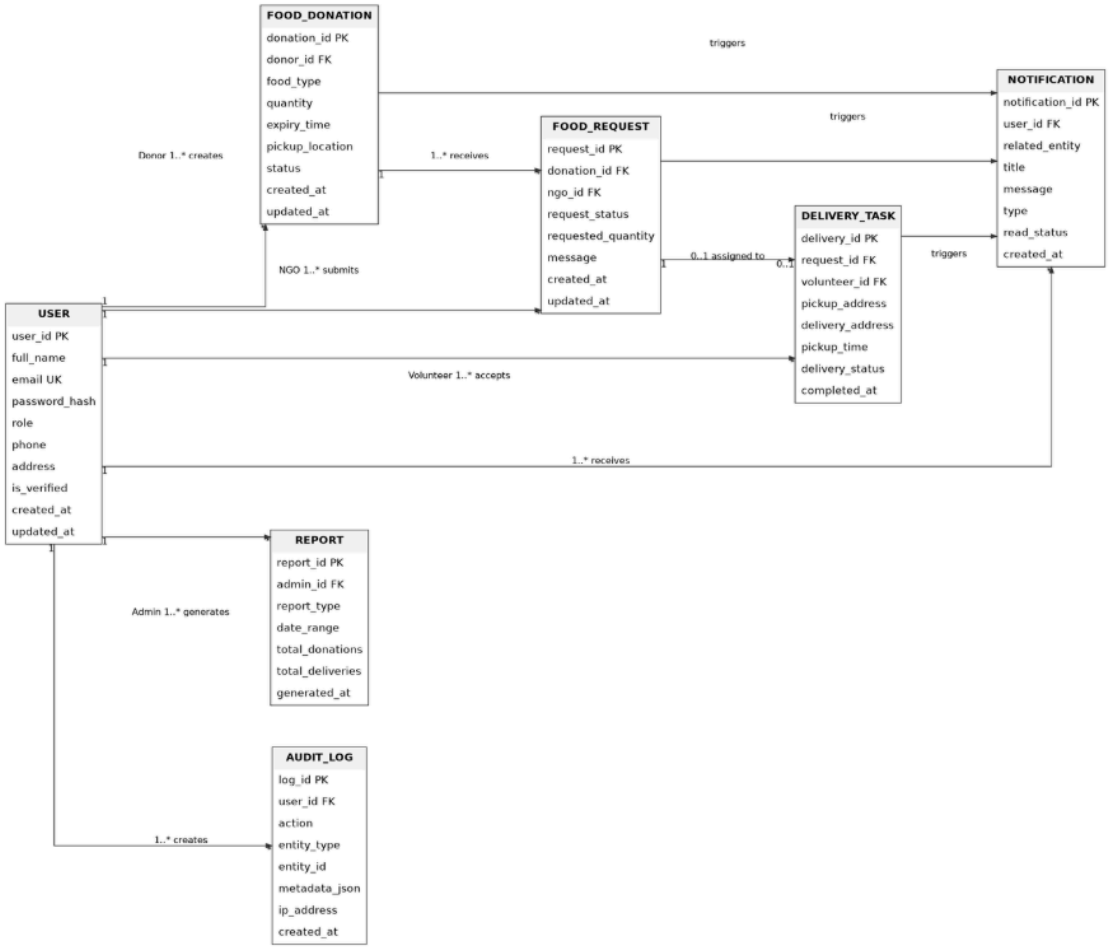
AuditLog:

log_id, user_id, action, entity_type, entity_id, metadata_json, ip_address, created_at

Relationship Rules:

1. One donor user can create many food donations.
2. One NGO user can submit many food requests.
3. One food donation can receive many food requests.
4. One approved food request can have one delivery task.
5. One volunteer can accept many delivery tasks.
6. Users can receive many notifications.
7. Administrators can generate many reports.
8. Audit logs record important user and system actions.

Mermaid ER Diagram:



Create System Design Document:

Description:

Create the system design document for FoodBridge. This document should explain system architecture, components, role-based access, workflow, security design, and deployment view.

Tasks:

1. Add system architecture diagram

2. Describe frontend, backend, database, notification, and reporting components

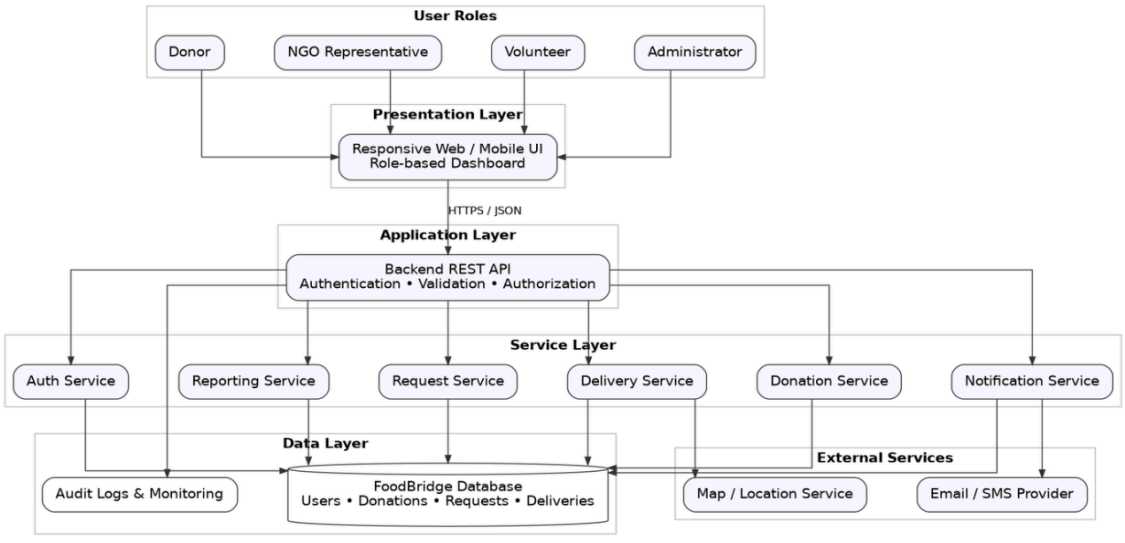
3 .Explain role-based access for donor, NGO, volunteer, and admin

4. Add high-level workflow

5. Add security and deployment design

System Architecture :

Food Bridge follows a layered, service-oriented modular architecture. Users interact through responsive role-based dashboards. The frontend communicates with a versioned backed REST API over HTTPS using JSON. The application layer performs authentication, input validation, authorization, and response mapping. Domain services implement workflow rules, repositories isolate DynamoDB access, and external adapters handle maps, notifications, storage, and monitoring.

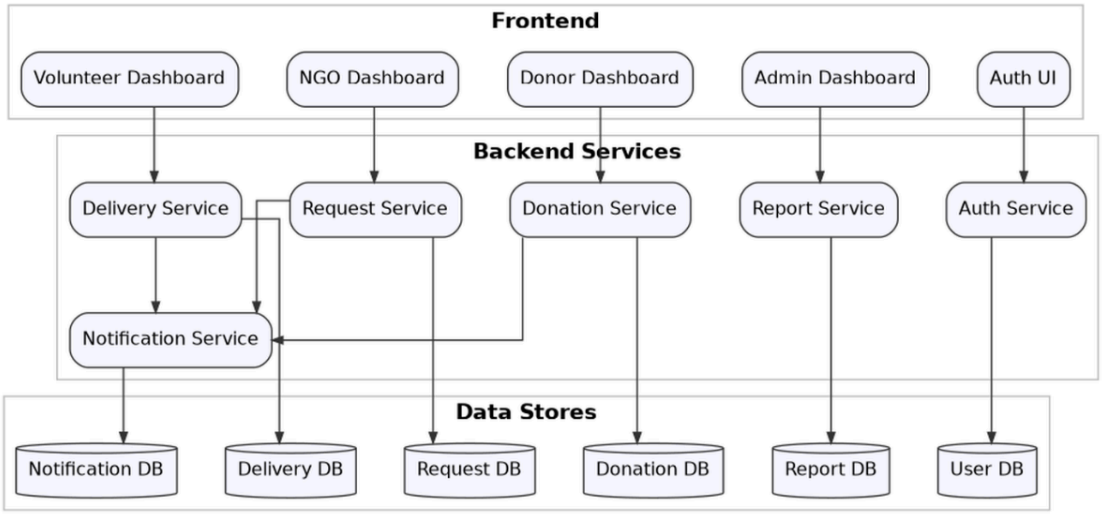


System Architecture – Food Bridge System

Component Diagram:

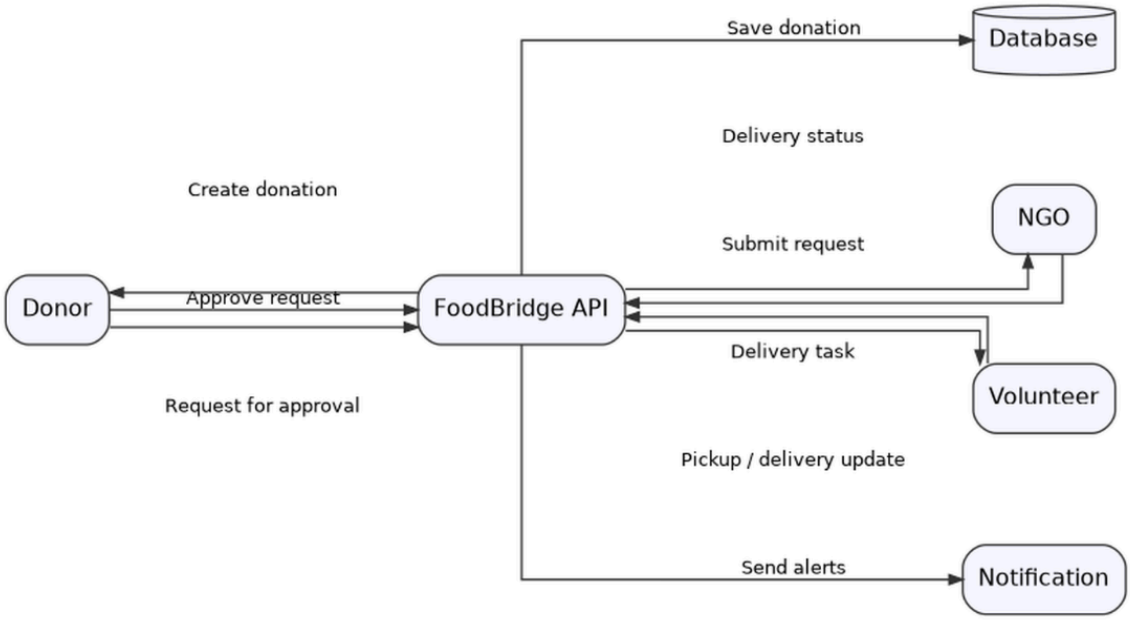
Component Responsibilities

Component	Primary Responsibility	Key Dependencies
Responsive Web UI	Authentication screens and role-based dashboards; forms; status timelines; validation feedback.	REST API; browser storage for non-sensitive preferences only.
Auth Service	Registration, login, token refresh, logout/revocation, password reset, account status checks.	User repository; token store/denylist; email provider.
Donation Service	Create, edit, publish, cancel, expire, browse, and track donations.	Donation repository; request service; notification service.
Request Service	Submit NGO requests, prevent duplicates, validate quantity, approve/reject requests.	Request and donation repositories; delivery service; notifications.
Delivery Service	Create/open task, volunteer acceptance, pickup, transit, delivery, receipt confirmation.	Delivery repository; map provider; notification service.
Notification Service	Persist in-app notifications; enqueue email/SMS; retry failed deliveries; track read state.	Notification repository; worker queue; provider adapters.
Reporting Service	Aggregate dashboard metrics and generate operational reports.	Repositories; object storage for generated files.
Audit and Monitoring	Record security/business events, trace requests, collect metrics, and support incident review.	Central logs, metrics, and alerting platform.



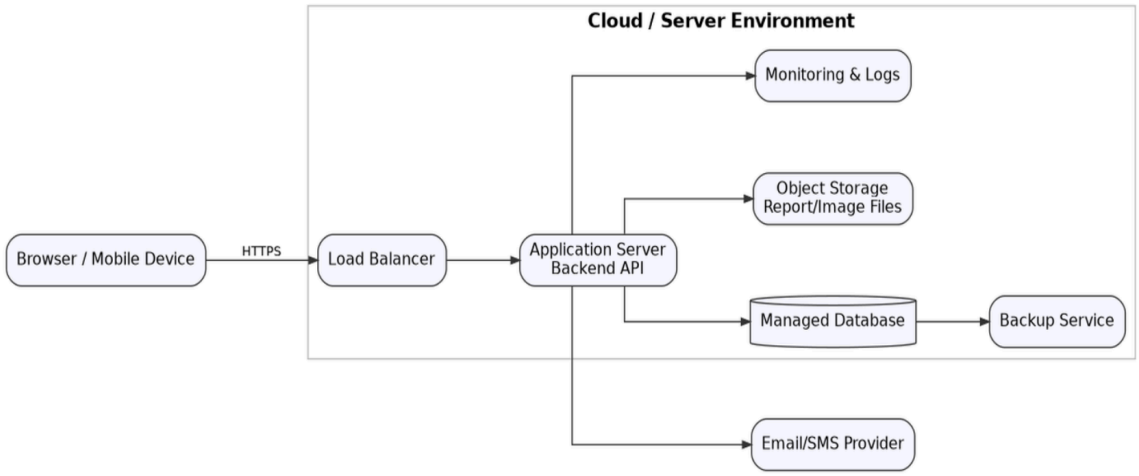
Component Diagram —Food Bridge System

DataFlow diagram:



DataFlow Diagram — Food Bridge System

Deployment Architecture:



Deployment Architecture– Food Bridge System

FoodBridge Technical Design Document :

Technical Stack

Layer	Technology
Frontend	React + TypeScript
Backend	Python FastAPI
Database	AWS DynamoDB
Authentication	JWT
Deployment	Docker on AWS

System Layers:

Controller Layer

Handles API requests, validation, authentication, and responses.

Responsibilities:

- Request parsing and validation

- Authentication dependency checks

- Role and ownership checks

- Response mapping and HTTP status codes

- Trace ID propagation

2. Service Layer

The service layer contains FoodBridge business rules:

- User registration and profile management

- NGO verification validation

- Food donation lifecycle management

- Donation expiration checks

- NGO request submission and duplicate prevention

- Donor request approval or rejection

- Volunteer delivery assignment

- Pickup, transit, delivery, and receipt confirmation

- Notification creation and delivery

- Dashboard and report aggregation

3. Repository Layer:

DynamoDB repositories use boto3 or a similar adapter.

- CRUD operations

- Query composition using PK, SK, and GSIs

- Conditional writes for state and ownership validation

- `TransactWriteItems` for multi-item consistency

- Cursor-based pagination

- Domain-model mapping

4. Authentication Layer:

- JWT access token issuance

- Refresh token rotation and revocation

- Role-based access control for Donor, NGO, Volunteer, and Administrator

- NGO verification guard before food request submission

- Account status validation on protected routes

5. Donation Service:

The Donation Service allows donors to create, update, publish, cancel, and track food donations.

Donation statuses:

DRAFT → AVAILABLE → REQUESTED → APPROVED → ASSIGNED → PICKED_UP → DELIVERED → COMPLETED

Alternative terminal states: CANCELLED,EXPIRED.

6. Food Request Service:

The Request Service allows verified NGOs to request suitable donations and allows donors to review them.

Request statuses:

PENDING → APPROVED → COMPLETED

Alternative terminal states: REJECTED,CANCELLED

7. Delivery Service:

The Delivery Service creates a task after request approval and manages volunteer delivery progress.

Delivery statuses:

OPEN → ASSIGNED → PICKED_UP → IN_TRANSIT → DELIVERED → CONFIRMED

Alternative terminal state: CANCELLED

8. Notification Service:

- Persists in-app notifications
- Sends email notifications through a provider adapter
- Tracks delivery status and retries
- Uses idempotency keys to reduce duplicate messages
- Generates notifications for:
  - New food request
  - Request approval/rejection
  - Volunteer assignment
  - Pickup confirmation
  - Delivery completion
  - NGO receipt confirmation

9. Admin and Reporting Service:

Administrators can:

- Manage user status

- Verify or reject NGO accounts

- Monitor donations, requests, and deliveries

- Review audit records

- Generate operational reports

- Track successful delivery rates and processing times

Main Modules

- User Management
- Donation Management
- Food Request Management
- Volunteer Delivery Management
- Notification Management
- Admin and Reporting

Security

- Password hashing
- JWT authentication
- Role-based authorization
- HTTPS communication
- NGO verification
- Audit logging

Scalability

- Stateless backend services
- Horizontal scaling
- DynamoDB indexes
- Background notification workers
- Cursor-based pagination

FoodBridge Database Design :

Database Overview

FoodBridge uses Amazon DynamoDB with a NoSQL single-table design.

Main Entities

Entity	Purpose
User	Stores account and role information
NGO Profile	Stores NGO verification details
Donation	Stores donated food information
Food Request	Stores NGO donation requests
Delivery	Stores volunteer delivery information
Notification	Stores user notifications
Audit Log	Stores system activity records
Report	Stores generated report information

Main Relationships

- One donor can create many donations.
- One donation can receive many NGO requests.
- One approved request creates one delivery task.
- One volunteer can manage many deliveries.
- Users can receive many notifications.
- Administrators can verify NGOs and monitor activities.

Important Database Fields

- `user_id`
- `donation_id`
- `request_id`
- `delivery_id`
- `food_type`

- `quantity`
- `expiration_at`
- `pickup_address`
- `status`
- `created_at`
- `updated_at`

Data Security

- Encryption at rest
- Least-privilege access
- Backup and recovery
- Audit logs
- Conditional database writes
- Soft deletion

FoodBridge API Design:

API Design

API Standards

Item	Standard
Base URL	**/api/v1**
Format	**JSON using UTF-8**
Authentication	**Bearer JWT access token**
Time format	**ISO-8601 UTC, e.g., 2026-06-20T10:30:00Z**
Identifiers	**Opaque UUID/ULID strings**
Pagination	**Cursor-based: cursor and limit; return next_cursor**

Idempotency	Idempotency-Key header for commands that may be retried
Tracing	X-Trace-ID request/response header
Versioning	Major version in URL; backward-compatible changes within a major version

Authentication API s

Method	Endpoint	Purpose	Access
POST	/auth/register	Register donor, NGO, or volunteer user.	Public
POST	/auth/login	Authenticate and issue access/refresh tokens.	Public
POST	/auth/refresh	Rotate refresh token and issue new access token.	Refresh session
POST	/auth/logout	Revoke refresh session.	Authenticated
POST	/auth/forgot-password	Start password reset flow.	Public
POST	/auth/reset-password	Complete password reset using a short-lived token.	Reset token

Donation and Request APIs

Method	Endpoint	Purpose	Role
POST	/donations	Create a donation.	Donor
GET	/donations	Browse available donations with filters and pagination.	Verified NGO / Admin
GET	/donations/{id}	View donation detail.	Authorized users

PATCH	/donations/{id}	Update an editable donation.	Owner Donor
DELETE	/donations/{id}	Cancel donation when permitted.	Owner Donor
POST	/donations/{id}/requests	Submit a food request.	Verified NGO
GET	/requests/mine	List current NGO requests.	NGO
POST	/requests/{id}/approve	Approve requests and create delivery tasks.	Owner Donor
POST	/requests/{id}/reject	Reject request with optional reason.	Owner Donor

Delivery, Notification, and Admin APIs

Method	Endpoint	Purpose	Role
GET	/deliveries/open	View eligible open delivery tasks.	Volunteer
POST	/deliveries/{id}/accept	Accept tasks using conditional updates.	Volunteer
POST	/deliveries/{id}/pickup	Confirm pickup.	Assigned Volunteer
POST	/deliveries/{id}/in-transit	Mark delivery in transit.	Assigned Volunteer
POST	/deliveries/{id}/deliver	Confirm delivery.	Assigned Volunteer
POST	/deliveries/{id}/confirm-receipt	Confirm NGO receipt.	Assigned NGO
GET	/notifications	List notifications.	Authenticated
PATCH	/notifications/{id}/read	Mark notification as read.	Owner
GET	/admin/users	List/manage users.	Admin

GET	/admin/ngos/pending	List pending NGO verifications.	Admin
POST	/admin/ngos/{id}/verify	Approve or reject NGO verification.	Admin
GET	/reports/dashboard	Return dashboard metrics.	Admin

Common Error Model

Editable JSON example

```
{
  "error": {
    "code": "INVALID_STATE_TRANSITION",
    "message": "The donation cannot be approved from its current status.",
    "details": {"current_status": "EXPIRED"},
    "trace_id": "01J..."
  }
}
```

HTTP Status	Use
400	Malformed or semantically invalid request.
401	Missing, invalid, or expired authentication.
403	Authenticated but not authorized.
404	Resource not found or not visible to caller.
409	Conflict, duplicate command, or invalid state transition.
422	Field-level validation error.
429	Rate limit exceeded.
500	Unexpected server error; return a trace ID without sensitive details.

Security Design

Control Area	Design
Authentication	**Short-lived JWT access tokens; rotating refresh tokens; secure password hashing; password reset tokens with expiry.**
Authorization	**Role, ownership, verification, account status, and resource-state checks on every protected command.**
Transport Security	**HTTPS/TLS only; secure headers; no sensitive data in URLs.**
Data Protection	**Encryption at rest, secrets manager, limited PII exposure, log redaction, and controlled exports.**
Input Security	**Schema validation, length/range checks, allow lists for status values, safe file upload checks, and output encoding.**
Abuse Protection	**Rate limits for login, registration, password reset, requests, and notification endpoints.**
Auditability	**Append-only audit events for authentication, verification, approvals, overrides, and status changes.**
Operational Security	**Least-privilege IAM, dependency scanning, container image scanning, patching, backups, and incident response.**

Security Event Examples

- Repeated failed login attempts or token reuse.
- NGO verification approval/rejection and document changes.
- Donation/request/delivery state override by an administrator.
- Change to user role, account status, or verification status.

Export or generation of reports containing operation